

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY

25 marks



Q2

CO4 - AT2 - CASE STUDY

25 marks



Q3

CO4 - AT2 - CASE STUDY

25 marks



Q4

CO4 - AT2 - CASE STUDY

25 marks



4/4 answered

Q1 CO4 - AT2 - CASE STUDY

25 marks

A large enterprise resource planning application is built from multiple source files, including modules for sales, inventory, and finance, and all of these modules must reference a common variable that stores the current fiscal year. The variable is defined once in one source file but must be read and, in some cases, modified by functions compiled separately in other source files. Without proper declaration, each module would either fail to compile or maintain its own inconsistent copy of the fiscal year. The design must ensure a single, consistent value is shared across the entire application.

Question: Identify which storage class allows the fiscal-year variable to be accessed across multiple source files, and explain the role of extern in achieving this shared access.

Your Code

```
1 #include <stdio.h>
2 int fiscal_year = 2024;
3 void sales_module_access();
4 #include <stdio.h>
5
6 // Global definition of the variable (usually in a main or config file)
7 int fiscal_year = 2024;
8
9 // Function prototypes
10 void sales_module_access();
11 void finance_module_modify();
12
13 int main() {
14     printf("--- ERP Application Startup ---\n");
15     printf("Initial Fiscal Year: %d\n", fiscal_year);
16
17     // Accessing the variable via the "Sales Module"
18     sales_module_access();
19
20     // Modifying the variable via the "Finance Module"
21     finance_module_modify();
22
23     printf("\nUpdated Fiscal Year after Finance Update: %d\n", fiscal_year);
24     printf("--- Application Shutdown ---\n");
25
26     return 0;
27 }
```

Input

stdin input

Output

Activate W
Go to Settings

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY
25 marks



Q2

CO4 - AT2 - CASE STUDY
25 marks



Q3

CO4 - AT2 - CASE STUDY
25 marks



Q4

CO4 - AT2 - CASE STUDY
25 marks



4/4 answered

Q2 CO4 - AT2 - CASE STUDY

25 marks

A smart utility meter installed in a residential complex can measure electricity consumption, water flow, or gas usage, but the hardware reports only one utility type per polling cycle depending on which sensor is currently active. Common information such as meter ID, timestamp, and location must always be recorded alongside whichever utility reading is captured. The billing server receives thousands of polling records daily and must interpret the correct reading type each time. Minimizing per-record memory footprint is a key design goal for the embedded firmware.

Question: Design a structure that embeds a union for the utility-specific reading, and explain why this hybrid representation is more memory-efficient than using three separate full-sized fields.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 typedef enum {
4     ELECTRICITY,
5     WATER,
6     GAS
7 } UtilityType;
8 typedef union {
9     float electricityKWh;
10    float waterLiters;
11    float gasCubicMeters;
12 } UtilityReading;
13 typedef struct {
14     int meterID;
15     char timestamp[20];
16     char location[30];
17     UtilityType type; // tells us which union member is valid
18     UtilityReading reading; // shared memory for the active reading
19 } MeterRecord;
20 void displayRecord(MeterRecord record) {
21     printf("Meter ID: %d\n", record.meterID);
22     printf("Timestamp: %s\n", record.timestamp);
23     printf("Location: %s\n", record.location);
24
25     switch (record.type) {
26         case ELECTRICITY:
27             printf("Electricity Reading: %.2f kWh\n", record.reading.electricityKWh);
28             break;
29         case WATER:
30             printf("Water Reading: %.2f Liters\n", record.reading.waterLiters);
31             break;
```

Input

stdin input

Output

QUESTIONS

Q1
CO4 - AT2 - CASE STUDY
25 marksQ2
CO4 - AT2 - CASE STUDY
25 marksQ3
CO4 - AT2 - CASE STUDY
25 marksQ4
CO4 - AT2 - CASE STUDY
25 marks

4/4 answered

Q3 CO4 - AT2 - CASE STUDY

25 marks

An insurance company processes claims for health, vehicle, and property policies, where each claim type requires distinct supporting information such as hospital details, accident details, or property-damage details respectively. Every claim, regardless of type, shares common fields such as claim ID, policyholder name, and claim date. Claims officers repeatedly retrieve and update a claim's status as it moves through approval stages. The company wants a design that avoids wasting memory on fields that do not apply to a given claim type. Question: Design a structure containing a union to represent claim-type-specific information, and explain how a structure pointer can be used to update a claim's status efficiently as it progresses through the workflow.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3
4 // Define types of claims
5 typedef enum { HEALTH, VEHICLE, PROPERTY } ClaimType;
6
7 // Union to hold specific details to save memory
8 union ClaimDetails {
9     struct {
10         char hospitalName[50];
11         float medicalBill;
12     } health;
13
14     struct {
15         char accidentLocation[50];
16         char vehicleModel[30];
17     } vehicle;
18
19     struct {
20         char propertyAddress[50];
21         float damageValue;
22     } property;
23 };
24
25 // Main structure representing a Claim
26 struct Claim {
27     int claimID;
28     char policyholderName[50];
29     char claimDate[15];
30     char status[20];
31     ClaimType type;
32     union ClaimDetails details;
```

Input

stdin input

Output

CO4 - AT2 - CASE STUDY

[← Back](#)

QUESTIONS

Q1

CO4 - AT2 - CASE STUDY
25 marks

Q2

CO4 - AT2 - CASE STUDY
25 marks

Q3

CO4 - AT2 - CASE STUDY
25 marks

Q4

CO4 - AT2 - CASE STUDY
25 marks

4/4 answered

Q4 CO4 - AT2 - CASE STUDY

25 marks

A distribution warehouse maintains inventory records for tens of thousands of products, where each record stores a product code, product name, quantity in stock, and supplier address consisting of several related sub-fields. Warehouse staff frequently search for a specific product to check stock levels and update the quantity after every incoming or outgoing shipment. Because the inventory is large, copying entire records during routine updates would be inefficient. The system must remain responsive even during high-volume shipment processing.

Question: Design an array of structures to hold the inventory, and explain how a structure pointer allows staff to update a selected product's stock quantity without copying the complete record.

Your Code

```
1 #include <stdio.h>
2 #include <string.h>
3 struct Address {
4     char street[50];
5     char city[30];
6     char state[20];
7 };
8 struct Product {
9     int productCode;
10    char productName[50];
11    int quantity;
12    struct Address supplierAddress;
13 };
14 void updateStock(struct Product *ptr, int newQuantity) {
15     ptr->quantity = newQuantity;
16 }
17 int main() {
18     struct Product inventory[3] = {
19         {101, "Laptop", 50, {"123 Tech Lane", "Chennai", "Tamil Nadu"}},
20         {102, "Mouse", 200, {"456 Click St", "Bangalore", "Karnataka"}},
21         {103, "Keyboard", 150, {"789 Type Ave", "Mumbai", "Maharashtra"}},
22     };
23     int searchCode, newQty;
24     int found = 0;
25     printf("Enter Product Code to update stock: ");
26     scanf("%d", &searchCode);
```

Input

stdin input

Output